

Databases (DBMS)

Complete MCQ-Oriented Notes

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

1. Introduction to DBMS	7
1.1 Data vs Information; Database; DBMS	7
1.2 File System vs DBMS (Drawbacks of File Processing)	7
1.3 Advantages of DBMS	8
1.4 Three-Schema (ANSI/SPARC) Architecture	8
1.5 Data Abstraction Levels	8
1.6 Data Independence	8
1.7 Database Users and DBA Roles	9
1.8 Data Models Overview	9
1.9 DBMS Components (Concept)	10
2. Entity-Relationship (ER) Model	11
2.1 Entities and Entity Sets	11
2.2 Attributes	11
2.3 Relationships and Relationship Sets	11
2.4 Degree of a Relationship	12
2.5 Mapping Cardinality	12
2.6 Participation Constraints	12
2.7 Keys (in ER)	12
2.8 Weak Entity Sets	13
2.9 Strong vs Weak Entities	13
3. Enhanced ER (EER) Model	14
3.1 Specialization and Generalization	14
3.2 Inheritance and Constraints	14
3.3 Aggregation	14
4. ER to Relational Mapping	14
4.1 Mapping Strong and Weak Entities	14
4.2 Mapping Relationships	15
4.3 Mapping Multivalued and Composite Attributes	15
4.4 Mapping Specialization/Generalization (Concept)	15
5. Relational Model Concepts	16
5.1 Relation, Tuple, Attribute, Domain, Degree, Cardinality	16
5.2 Relation Schema vs Instance	16
5.3 Properties of Relations	16

5.4 NULL Values and Three-Valued Logic	16
5.5 Keys (Relational)	17
5.6 Integrity Constraints	17
5.7 Referential Actions	17
6. Relational Algebra (Procedural)	19
6.1 Selection, Projection, Rename	19
6.2 Set Operations and Union-Compatibility	19
6.3 Cartesian Product	19
6.4 Joins	19
6.5 Division Operation	20
6.6 Query Equivalence (Concept)	20
7. Relational Calculus (Declarative)	20
7.1 Tuple Relational Calculus (TRC)	20
7.2 Domain Relational Calculus (DRC)	20
7.3 Safe Expressions	21
7.4 Equivalence of Algebra and Calculus	21
8. SQL - Data Definition Language (DDL)	22
8.1 SQL Data Types	22
8.2 CREATE TABLE and Constraints	22
8.3 ALTER, DROP, TRUNCATE	22
8.4 Creating Indexes	23
9. SQL - Data Manipulation Language (DML)	24
9.1 SELECT, DISTINCT, Aliases	24
9.2 WHERE Clause Operators	24
9.3 ORDER BY	24
9.4 INSERT, UPDATE, DELETE	24
9.5 Joins in SQL	24
9.6 Set Operations in SQL	25
10. SQL - Aggregation and Grouping	26
10.1 Aggregate Functions	26
10.2 COUNT(*) vs COUNT(column)	26
10.3 GROUP BY	26
10.4 HAVING vs WHERE	26
10.5 Logical Order of Clause Evaluation	26

11. SQL - Subqueries	26
11.1 Subqueries in WHERE / FROM / SELECT	27
11.2 IN, NOT IN, EXISTS, NOT EXISTS	27
11.3 ANY/SOME and ALL	27
11.4 Correlated Subqueries	27
11.5 NOT IN with NULL Pitfall	27
12. SQL - Other Objects	27
12.1 Views	27
12.2 Triggers	27
12.3 Stored Procedures / Functions	28
12.4 GRANT / REVOKE (DCL)	28
13. Functional Dependencies (FD)	29
13.1 Definition and Notation	29
13.2 Trivial vs Non-Trivial FDs	29
13.3 Armstrong's Axioms	29
13.4 Closure of an Attribute Set (X ⁺)	29
13.5 Finding Candidate Keys from FDs	30
13.6 Prime vs Non-Prime Attributes	30
13.7 Minimal / Canonical Cover	30
14. Normal Forms	31
14.1 Anomalies	31
14.2 First Normal Form (1NF)	31
14.3 Second Normal Form (2NF)	31
14.4 Third Normal Form (3NF)	31
14.5 BCNF (Boyce-Codd Normal Form)	31
14.6 Determining the Highest Normal Form	32
14.7 4NF and Multivalued Dependencies (Awareness)	32
15. Decomposition	32
15.1 Lossless Join Decomposition	32
15.2 Dependency-Preserving Decomposition	32
15.3 3NF vs BCNF Trade-off	32
16. File Organization	34
16.1 Records: Fixed vs Variable Length	34
16.2 Heap (Unordered) Organization	34
16.3 Sequential (Ordered) Organization	34

16.4 Hash File Organization	34
16.5 Blocking Factor / Block Concept	34
17. Indexing	34
17.1 Index Concept and Search Key	34
17.2 Primary (Clustering) vs Secondary (Non-Clustering)	34
17.3 Dense vs Sparse Index	35
17.4 Multilevel Index	35
17.5 B-Tree	35
17.6 B+ Tree	35
17.7 Hashing for Indexing; Extendible Hashing (Awareness)	36
18. Transactions	37
18.1 Transaction Concept; Read/Write Operations	37
18.2 ACID Properties	37
18.3 Transaction States	37
18.4 Serial vs Concurrent Schedules	37
18.5 Need for Concurrency	38
19. Concurrency Problems	38
19.1 Lost Update	38
19.2 Dirty Read	38
19.3 Unrepeatable Read	38
19.4 Phantom Read	38
20. Serializability	39
20.1 Conflict Operations and Conflict Serializability	39
20.2 Precedence (Serialization) Graph	39
20.3 View Serializability	39
20.4 Recoverability	39
21. Concurrency Control Protocols	40
21.1 Lock-Based Protocols	40
21.2 Deadlock Handling	40
21.3 Timestamp Ordering Protocol	41
21.4 Optimistic / Validation-Based	41
22. Recovery	42
22.1 Failure Types	42
22.2 Log-Based Recovery; Deferred vs Immediate Update	42

22.3 Checkpoints; Write-Ahead Logging	42
22.4 Shadow Paging (Awareness)	42
23. NoSQL Databases (Dedicated Section)	43
23.1 What is NoSQL and Why It Emerged	43
23.2 Key-Value Stores	43
23.3 Document Stores	43
23.4 Column-Family (Wide-Column) Stores	43
23.5 Graph Databases	44
23.6 CAP Theorem (Brewer's Theorem)	44
23.7 BASE vs ACID	44
23.8 Sharding (Horizontal Partitioning)	45
23.9 Replication	45
23.10 SQL vs NoSQL Summary	46
24. CIL MT Exam Focus / Practice Checklist	47
24.1 Candidate Key Finding from FD Sets	47
24.2 Identifying the Highest Normal Form	47
24.3 Lossless Decomposition Test	47
24.4 Conflict-Serializability via Precedence Graph	47
24.5 SQL Query Output / Equivalence	47
24.6 Relational Algebra <-> SQL Translation	47
24.7 B/B+ Tree Order, Height, Node-Count	47
24.8 Counting Tuples in Joins/Set Operations	47
24.9 Definitions and One-Liners	47

1. Introduction to DBMS

1.1 Data vs Information; Database; DBMS

- **Data** are raw, unprocessed, isolated facts and figures (e.g., a number 42, a name Ram) that have no meaning by themselves.
- **Information** is processed, organized, and structured data that carries meaning and supports decision-making (e.g., "Ram scored 42 marks").
- The transformation chain is **Data -> Processing -> Information -> Knowledge -> Wisdom** (the DIKW pyramid).
- A **database** is a **shared, organized collection of logically related data** stored persistently and designed to meet the information needs of an organization.
- A **DBMS (Database Management System)** is the **software** that enables users to **define, create, maintain, query, and control** access to a database (examples: **Oracle, MySQL, PostgreSQL, SQL Server, DB2**).
- The set **DBMS + database + applications + users** together is called a **Database System**.
- A **metadata** is "data about data" — the schema/structure description stored in the **data dictionary (system catalog)**.

Memory Trick

- **"DIKW"** = Data, Information, Knowledge, Wisdom (the value-addition ladder).

CBT Trap

- Examiners equate "database" with "DBMS"; remember the **database is the data**, the **DBMS is the software** that manages it — they are NOT the same.
- **Metadata** is stored in the **data dictionary / system catalog**, NOT in user tables.

1.2 File System vs DBMS (Drawbacks of File Processing)

- A **file-processing system** stores data in **separate OS files**, each accessed by application-specific programs.
- Major drawbacks of file systems that DBMS solves:
- **Data redundancy** — same data duplicated in many files wasting storage.
- **Data inconsistency** — duplicate copies become out of sync.
- **Difficulty in accessing data** — every new query needs a new program.
- **Data isolation** — data scattered across files/formats.
- **Integrity problems** — constraints hard-coded in programs, hard to change.
- **Atomicity problems** — partial updates on crash leave data inconsistent.
- **Concurrent-access anomalies** — uncontrolled simultaneous updates corrupt data.
- **Security problems** — hard to enforce fine-grained access control.

Aspect	File System	DBMS
Redundancy	High	Controlled/minimized
Consistency	Hard to maintain	Enforced via constraints
Concurrency	Poor	Managed by concurrency control
Security	Coarse (file level)	Fine (table/row/column)
Querying	Program needed	Declarative (SQL)
Data independence	None	Logical & physical

CBT Trap

- "Data redundancy" (duplication) is the **cause**; "data inconsistency" is the **effect**. Questions often ask which one DBMS reduces — the answer is it **controls redundancy**, which prevents inconsistency.

1.3 Advantages of DBMS

- **Controlled redundancy** — store each fact once (not necessarily zero redundancy; some is kept for performance).
- **Consistency** — constraints keep data valid.
- **Integrity** — rules (domain, key, referential) enforced centrally.
- **Data sharing** — multiple users/applications use the same data concurrently.
- **Enforced standards, backup and recovery, security/authorization, and data independence.**
- **Reduced application development time** due to high-level query languages.

CBT Trap

- DBMS does **NOT eliminate** redundancy entirely — it **controls/minimizes** it. "Zero redundancy" is a wrong option.

1.4 Three-Schema (ANSI/SPARC) Architecture

- Proposed by **ANSI/SPARC** in **1975**; it defines **3** levels to separate the user view from physical storage.
- **External level (View level)** — many **user views**, each showing the part of the DB relevant to a user group; hides the rest.
- **Conceptual level (Logical level)** — a single **community/logical schema** describing entities, relationships, constraints for the whole enterprise.
- **Internal level (Physical level)** — describes **physical storage**: files, indexes, access paths, data structures.
- Two mappings connect them: **external/conceptual mapping** and **conceptual/internal mapping**.

Level	Also Called	Describes	Mapping Above
External	View	What each user sees	external/conceptual
Conceptual	Logical	Whole enterprise schema	conceptual/internal
Internal	Physical	Storage, files, indexes	—

Memory Trick

- **"ECI"** top-to-bottom = External, Conceptual, Internal (user -> logical -> storage).

CBT Trap

- **View level** = external = topmost (closest to user). **Physical level** = internal = bottom. Don't swap them.

1.5 Data Abstraction Levels

- **Physical level** — lowest; HOW data is stored (data structures, byte layout).
- **Logical level** — WHAT data is stored and the relationships among data.
- **View level** — highest; shows only part of the DB, simplifying user interaction and providing security.
- Abstraction hides complexity: users at the view level need not know storage details.

CBT Trap

- The **3** abstraction levels map one-to-one to the **3** schema levels: physical=internal, logical=conceptual, view=external.

1.6 Data Independence

- **Data independence** = capacity to change the schema at one level without changing it at the higher level.
- **Logical data independence** — ability to change the **conceptual schema** (add/remove attributes, relations) without changing **external schemas/applications**.
- **Physical data independence** — ability to change the **internal schema** (storage, indexes) without changing the **conceptual schema**.
- **Physical** data independence is **easier** to achieve; **logical** data independence is **harder** because applications depend heavily on the logical structure.

Memory Trick

- "**Logical is harder than Physical**" — both 'L' and 'Harder' indicate the tougher one.

CBT Trap

- Adding an index affects only the internal level => this tests **physical** data independence. Adding a new column tests **logical** data independence.

1.7 Database Users and DBA Roles

- **Naive/parametric users** — use canned application interfaces (e.g., bank teller).
- **Application programmers** — write programs using a host language + SQL.
- **Sophisticated users** — analysts who write ad-hoc complex queries.
- **Specialized users** — build special-purpose DB applications (CAD, GIS).
- **DBA (Database Administrator)** — central control; responsibilities:
- **Schema definition, storage structure and access-method definition.**
- **Granting authorization** (GRANT/REVOKE) and security.
- **Routine maintenance:** backups, performance tuning, capacity planning.
- **Integrity-constraint specification.**

CBT Trap

- **GRANT/REVOKE** (authorization) is a **DBA** duty, not an application programmer's.

1.8 Data Models Overview

- A **data model** is a collection of conceptual tools for describing data, relationships, semantics, and constraints.
- **Hierarchical model** — tree structure; **1:N** parent-child; e.g., **IBM IMS**.
- **Network model** — graph structure; supports **M:N** via sets; CODASYL/DBTG.
- **Relational model** — data in **tables (relations)**; proposed by **E.F. Codd in 1970**.
- **ER model** — high-level conceptual model using entities and relationships (Chen, **1976**).
- **Object-oriented** and **object-relational** models — objects, inheritance.

Model	Structure	Relationship	Example
Hierarchical	Tree	1:N only	IBM IMS
Network	Graph	M:N	CODASYL
Relational	Tables	via keys	Oracle, MySQL
ER	Diagram	concept	design tool

CBT Trap

- **Relational model** was given by **E.F. Codd (1970)**; **ER model** by **Peter Chen (1976)**. Don't confuse the two names/years.

1.9 DBMS Components (Concept)

- **Query processor:** DDL interpreter, DML compiler (+ query optimizer), query evaluation engine.
- **Storage manager:** authorization & integrity manager, transaction manager, file manager, buffer manager.
- **Disk storage:** data files, data dictionary, indices, statistical data.
- The **buffer manager** brings disk blocks into main memory — central to performance.

CBT Trap

- The **query optimizer** chooses the **lowest-cost** execution plan; it is part of the **DML compiler/query processor**, not the storage manager.

2. Entity-Relationship (ER) Model

2.1 Entities and Entity Sets

- An **entity** is a "thing" or object in the real world distinguishable from others (e.g., a specific student Ram).
- An **entity set** is a **collection of entities of the same type** sharing the same attributes (e.g., all Students).
- In an ER diagram an entity set is drawn as a **rectangle**.
- **Entity type** is the schema (definition); **entity set** is the current collection of instances.

CBT Trap

- An "entity" is one instance; an "entity set" is the whole collection — MCQs swap these terms.

2.2 Attributes

- An **attribute** is a property describing an entity; drawn as an **ellipse** connected to its entity rectangle.

2.2.1 Simple vs Composite

- **Simple (atomic)** attribute cannot be divided (e.g., Age).
- **Composite** attribute can be split into sub-parts (e.g., Name -> first, middle, last; Address -> street, city, pin).

2.2.2 Single-valued vs Multivalued

- **Single-valued** holds one value per entity (e.g., DateOfBirth).
- **Multivalued** holds a set of values (e.g., PhoneNumbers); drawn as a **double ellipse**.

2.2.3 Stored vs Derived

- **Stored** attribute is physically stored (e.g., DateOfBirth).
- **Derived** attribute is computed from others (e.g., Age from DateOfBirth); drawn as a **dashed ellipse**.

2.2.4 Key Attribute; NULL Attributes

- **Key attribute** uniquely identifies an entity; its ellipse text is **underlined**.
- **NULL** value means **unknown**, **not applicable**, or **missing**.

Attribute Type	ER Symbol
Simple/single-valued	Single ellipse
Key	Underlined ellipse
Multivalued	Double ellipse
Derived	Dashed ellipse
Composite	Ellipse with sub-ellipses

Memory Trick

- "**Double = multi**, **Dashed = derived**, **Underline = key**".

CBT Trap

- **Derived** = **dashed** ellipse; **Multivalued** = **double** ellipse. Easy to swap — derived is dashed (not solid double).

2.3 Relationships and Relationship Sets

- A **relationship** is an association among two or more entities (e.g., Ram borrows Book#101).

- A **relationship set** is a set of relationships of the same type; drawn as a **diamond**.
- Relationships may have **descriptive attributes** (e.g., date on a borrows relationship).

2.4 Degree of a Relationship

- **Degree** = number of entity sets participating.
- **Unary (recursive), degree 1** — entity relates to itself (e.g., Employee manages Employee).
- **Binary, degree 2** — most common (e.g., Student enrolls Course).
- **Ternary, degree 3** — three entity sets (e.g., Supplier-Part-Project).

CBT Trap

- A **recursive (self) relationship** has degree **1 (unary)**, even though it visually connects "two ends" of the same entity set.

2.5 Mapping Cardinality

- **Mapping cardinality** expresses how many entities of one set associate with another.
- **1:1** — one-to-one (e.g., Employee manages one Department, each Dept one manager).
- **1:N** — one-to-many (e.g., one Department has many Employees).
- **M:N** — many-to-many (e.g., Students and Courses).

Cardinality	Example
1:1	Person : Passport
1:N	Department : Employee
M:N	Student : Course

CBT Trap

- Cardinality ratio is decided by the **maximum** number of associations, not the minimum (which is participation).

2.6 Participation Constraints

- **Participation** expresses whether **every** entity must participate.
- **Total participation** — every entity **MUST** participate (drawn as **double line**); existence-dependent.
- **Partial participation** — some entities may not participate (single line).

CBT Trap

- **Total participation** = **double line**; it sets the **minimum cardinality to 1**. Cardinality ratio (max) and participation (min) are different constraints.

2.7 Keys (in ER)

- **Super key** — any set of attributes that uniquely identifies an entity (may have extra attributes).
- **Candidate key** — a **minimal** super key (no proper subset is a super key).
- **Primary key** — the candidate key chosen by the designer; **unique + NOT NULL**.
- **Alternate key** — candidate keys not chosen as primary.

Memory Trick

- "**Super >= Candidate >= Primary**" (each is a subset relationship of strictness).

CBT Trap

- Every **candidate key** is a **super key**, but not every super key is a candidate key (candidate must be **minimal**).

2.8 Weak Entity Sets

- A **weak entity set** has **no primary key of its own**; it depends on a **strong (owner) entity**.
- It is drawn as a **double rectangle**; its connecting **identifying relationship** is a **double diamond**.
- A **partial key (discriminator)** distinguishes weak entities of the same owner; drawn as a **dashed underline**.
- A weak entity always has **total participation** in its identifying relationship.
- Full identifier of a weak entity = **owner's primary key + partial key**.

CBT Trap

- A weak entity's identifier is the combination of **owner PK + its own partial key** — NOT the partial key alone.

2.9 Strong vs Weak Entities

Feature	Strong Entity	Weak Entity
Primary key	Has its own	Lacks own (partial key)
Symbol	Single rectangle	Double rectangle
Existence	Independent	Dependent on owner
Participation in ID rel	—	Total

3. Enhanced ER (EER) Model

3.1 Specialization and Generalization

- **Specialization** is **top-down**: split a higher entity into specialized **subclasses** (e.g., Employee -> Engineer, Manager).
- **Generalization** is **bottom-up**: combine lower entities into a higher **superclass** (e.g., Car, Truck -> Vehicle).
- Represented by an **ISA triangle** connecting subclass to superclass.

Memory Trick

- "**Specialize = Split (top-down); Generalize = Group (bottom-up)**".

CBT Trap

- **Generalization** = bottom-up (combine); **Specialization** = top-down (divide). They are inverse processes; examiners flip them.

3.2 Inheritance and Constraints

- Subclasses **inherit** all attributes and relationships of the superclass (attribute + relationship inheritance).
- **Disjoint (d)** constraint — an entity belongs to **at most one** subclass.
- **Overlapping (o)** constraint — an entity may belong to **multiple** subclasses.
- **Total** specialization — every superclass member must belong to **some** subclass (double line).
- **Partial** specialization — a member need not belong to any subclass.

Constraint Pair	Meaning
Disjoint vs Overlapping	one subclass vs many
Total vs Partial	must belong vs may not

CBT Trap

- **Disjoint** controls "how many subclasses per entity"; **Total/partial** controls "must every entity be in a subclass". They are independent dimensions.

3.3 Aggregation

- **Aggregation** treats a **relationship (and its entities) as a higher-level entity**, allowing relationships among relationships.
- Used when a relationship must itself participate in another relationship (e.g., a works-on relationship being managed).

CBT Trap

- Aggregation is the EER tool used to model a **relationship participating in another relationship** — ternary relationships are an alternative but not identical.

4. ER to Relational Mapping

4.1 Mapping Strong and Weak Entities

- **Strong entity** -> a **table** with all simple attributes; primary key = entity's key.

- **Weak entity** -> a table including the **owner's primary key as a foreign key**; PK = **owner PK + partial key**.

4.2 Mapping Relationships

- **1:1** -> add the PK of one side as a **foreign key** in the other (prefer the side with total participation).
- **1:N** -> put the PK of the "1" side as a **foreign key** in the "N" side table. No new table needed.
- **M:N** -> create a **new relationship (junction) table** whose PK = **combination of both PKs** + any descriptive attributes.

Relationship	Mapping Rule	New Table?
1:1	FK on either side	No
1:N	FK on N-side	No
M:N	Separate junction table	Yes

CBT Trap

- Only **M:N** relationships **require a separate table**. For **1:N**, NEVER create a new table — put the FK on the many side.

4.3 Mapping Multivalued and Composite Attributes

- **Multivalued** attribute -> a **separate table** containing the entity's PK + the multivalued attribute; PK = both together.
- **Composite** attribute -> create **one column per simple sub-component** (the composite itself is not stored as a single column).

CBT Trap

- A **multivalued** attribute always forces a **new table**; a **composite** attribute does NOT — it just becomes multiple columns.

4.4 Mapping Specialization/Generalization (Concept)

- **Method 1**: separate tables for superclass and each subclass (subclass table holds PK + its specific attributes).
- **Method 2**: a table per subclass containing all inherited + specific attributes (no superclass table).
- **Method 3**: single table for the whole hierarchy with a **type/discriminator** attribute (many NULLs).

CBT Trap

- The **single-table** approach causes many **NULL** values for inapplicable subclass attributes — a common disadvantage asked in exams.

5. Relational Model Concepts

5.1 Relation, Tuple, Attribute, Domain, Degree, Cardinality

- A **relation** is a **table** with rows and columns; formally a subset of the Cartesian product of domains.
- A **tuple** is a **row** of the relation (a single record).
- An **attribute** is a **column** (named).
- A **domain** is the set of **allowed atomic values** for an attribute.
- **Degree (arity)** = number of **attributes (columns)**.
- **Cardinality** = number of **tuples (rows)**.

Term	Meaning
Relation	Table
Tuple	Row
Attribute	Column
Domain	Allowed value set
Degree	# columns
Cardinality	# rows

Memory Trick

- "**Degree = columns (Definition of schema), Cardinality = Count of rows**".

CBT Trap

- **Degree** = number of **columns**; **Cardinality** = number of **rows**. Frequently swapped in MCQs.

5.2 Relation Schema vs Instance

- **Relation schema** = the **structure**: relation name + attributes + domains, e.g., Student(Roll, Name, Age). It is **time-invariant**.
- **Relation instance** = the **actual set of tuples** at a given moment; it **changes over time**.

CBT Trap

- Schema is the "intension" (fixed design); instance is the "extension" (current data). Don't confuse intension with extension.

5.3 Properties of Relations

- **No duplicate tuples** — a relation is a **set** of tuples (pure relational model).
- **Tuples are unordered** (no top-to-bottom order).
- **Attributes are unordered** (identified by name, not position).
- All values are **atomic (single-valued)** — satisfies 1NF.
- **Attribute names are distinct** within a relation.

CBT Trap

- In the pure relational model there are **no duplicate rows** and **no ordering**; but SQL tables are **multisets (bags)** and **allow duplicates** unless constrained. This SQL-vs-theory difference is a classic trap.

5.4 NULL Values and Three-Valued Logic

- **NULL** represents **unknown**, **inapplicable**, or **missing** data; it is **not** equal to 0 or empty string.
- SQL uses **three-valued logic (3VL)**: **TRUE**, **FALSE**, **UNKNOWN**.
- Any comparison with NULL (e.g., NULL = NULL, NULL > 5) yields **UNKNOWN**; use **IS NULL** / **IS NOT NULL**.
- UNKNOWN AND FALSE = FALSE; UNKNOWN OR TRUE = TRUE; NOT UNKNOWN = UNKNOWN.

CBT Trap

- WHERE keeps rows only when the predicate is **TRUE** (not UNKNOWN). NULL = NULL is **UNKNOWN**, so such rows are dropped.

5.5 Keys (Relational)

- **Super key** — set of attributes uniquely identifying a tuple.
- **Candidate key** — minimal super key.
- **Primary key** — chosen candidate key; **unique + NOT NULL**.
- **Foreign key** — attribute(s) referencing the PK of another (or same) relation.
- **Composite key** — a key consisting of **>= 2 attributes**.

CBT Trap

- A **foreign key** CAN be **NULL** (unless declared NOT NULL); a **primary key** can **never** be NULL (entity integrity).

5.6 Integrity Constraints

5.6.1 Domain Constraint

- Each attribute value must be an **atomic value from its domain** (correct type/range).

5.6.2 Key Constraint

- **No two tuples** may have the **same value** for the candidate/primary key (uniqueness).

5.6.3 Entity Integrity

- The **primary key cannot be NULL** (no part of a composite PK may be NULL).

5.6.4 Referential Integrity

- A **foreign key** must either **match an existing primary key value** in the referenced relation or be **NULL**.

Constraint	Rule
Domain	Atomic value from domain
Key	Uniqueness of key
Entity integrity	PK not NULL
Referential	FK matches PK or is NULL

CBT Trap

- **Entity integrity** -> about **PK NOT NULL**; **Referential integrity** -> about **FK matching a PK**. Don't interchange them.

5.7 Referential Actions

- On delete/update of a referenced (parent) row, the action on child rows can be:
- **CASCADE** — propagate the delete/update to child rows.

- **SET NULL** — set the FK in child rows to NULL.
- **SET DEFAULT** — set FK to a default value.
- **RESTRICT / NO ACTION** — reject the operation if children exist.

CBT Trap

- **ON DELETE CASCADE** deletes child rows automatically; **RESTRICT** blocks the parent delete. Picking the wrong effect is a common error.

6. Relational Algebra (Procedural)

- **Relational algebra** is a **procedural** query language: it specifies **how** to compute the result via a sequence of operators on relations; output of each operator is again a relation (closure property).
- **Basic (primitive) operators**: there are **6** — **Selection**, **Projection**, **Union**, **Set difference**, **Cartesian product**, and **Rename**.
- **Derived operators**: **Intersection**, **Join** (all kinds), **Division**, **Assignment**.

6.1 Selection, Projection, Rename

- **Selection (sigma)** $\sigma_{\text{predicate}}(R)$ — picks **rows (tuples)** satisfying a condition; it is a **horizontal** subset. Degree unchanged; cardinality $\leq$ original.
- **Projection (pi)** $\pi_{\text{attrs}}(R)$ — picks **columns**; a **vertical** subset. In pure algebra it **removes duplicates** (set semantics).
- **Rename (rho)** $\rho_S(R)$ — renames a relation (and/or its attributes), useful for self-joins.
- **Selection is commutative**: $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$.

Operator	Symbol	Acts on
Selection	σ (sigma)	Rows
Projection	π (pi)	Columns
Rename	ρ (rho)	Names

Memory Trick

- "**Sigma = Select rows (horizontal); Pi = Project columns (vertical)**".

CBT Trap

- **Projection in relational algebra removes duplicates** (set), but SQL **SELECT** keeps duplicates unless **DISTINCT** is used. A favorite count-the-tuples trap.

6.2 Set Operations and Union-Compatibility

- **Union ($\cup$)**, **Intersection ($\cap$)**, and **Difference ($-$)** require the two relations to be **union-compatible**: **same degree** AND **corresponding attributes from the same domains**.
- $R \cup S$ = tuples in R or S (duplicates removed).
- $R \cap S$ = tuples in both; $R \cap S = R - (R - S)$.
- $R - S$ = tuples in R but not S; difference is **NOT commutative**.

CBT Trap

- **Cartesian product** and **Join** do **NOT** need union-compatibility; only $\cup$, $\cap$, $-$ do. Also $R - S \neq S - R$.

6.3 Cartesian Product

- **Cartesian product ($\times$)** $R \times S$ pairs every tuple of R with every tuple of S.
- If $|R| = m$ rows, degree a, and $|S| = n$ rows, degree b: result has **$m \times n$ rows** and **$a+b$ columns**.

CBT Trap

- Cross product **multiplies** row counts and **adds** column counts: rows = $m \times n$, degree = $a+b$.

6.4 Joins

- **Theta join** $R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$ with any condition θ .
- **Equijoin** — theta join where θ uses only **equality (=)**; join columns appear **twice**.
- **Natural join** ($\bowtie$) — equijoin on **all common attributes**, then removes the duplicate column(s). If no common attribute, it becomes a **Cartesian product**.
- **Outer joins** preserve unmatched tuples, padding with **NULL**:
- **Left outer join** ($\bowtie\leftarrow$) keeps all left tuples.
- **Right outer join** ($\rightarrow\bowtie$) keeps all right tuples.
- **Full outer join** ($\bowtie\leftrightarrow$) keeps all tuples from both.

Join	Keeps Unmatched
Inner / Natural	None
Left outer	Left only
Right outer	Right only
Full outer	Both

CBT Trap

- **Natural join with no common attribute = Cartesian product**. Also, equijoin keeps both join columns while natural join keeps **one**.

6.5 Division Operation

- **Division** ($\div$) $R \div S$ answers **"for all"** queries: returns tuples of R associated with **every** tuple of S.
- Schema of result = attributes of R **not in** S.
- Example: "students enrolled in **all** courses" = Enrolled $\div$ Courses.

CBT Trap

- **Division** is the operator for **"for all"/universal** queries; relate it to a **double negation** using NOT EXISTS in SQL.

6.6 Query Equivalence (Concept)

- Different algebra expressions can yield the same result; the **query optimizer** uses equivalence rules (push selections, reorder joins) to reduce cost.
- **Selection pushdown** (apply σ as early as possible) reduces intermediate sizes.

7. Relational Calculus (Declarative)

- **Relational calculus** is a **non-procedural (declarative)** language: specifies **what** to retrieve, not how. Based on **first-order predicate logic**.

7.1 Tuple Relational Calculus (TRC)

- Variables range over **tuples**. Form: $\{ t \mid P(t) \}$ — set of tuples t for which predicate P is true.
- Quantifiers: **existential** $\exists$ ("there exists") and **universal** $\forall$ ("for all").
- Example: $\{ t \mid t \in \text{Student} \wedge t.\text{Age} > 20 \}$.

7.2 Domain Relational Calculus (DRC)

- Variables range over **domains (attribute values)**, not whole tuples.
- Form: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, \dots, x_n) \}$.

- **QBE (Query By Example)** is based on DRC.

7.3 Safe Expressions

- A **safe expression** produces a **finite** result whose values come from the **domain of the database** (active domain), avoiding infinite results like $\{ t \mid \text{NOT}(t \in R) \}$.

7.4 Equivalence of Algebra and Calculus

- **Codd's theorem**: **relational algebra**, **safe TRC**, and **safe DRC** are **equivalent in expressive power (relationally complete)**.

CBT Trap

- Relational algebra is **procedural**; relational calculus is **declarative** — yet they have **equal expressive power** (Codd's theorem). Calculus is NOT more powerful.

8. SQL - Data Definition Language (DDL)

- **SQL = Structured Query Language**, based on relational algebra + calculus; originally **SEQUEL** by IBM. It is **declarative**.
- SQL sub-languages: **DDL** (define), **DML** (manipulate), **DCL** (GRANT/REVOKE), **TCL** (COMMIT/ROLLBACK/SAVEPOINT), **DQL** (SELECT).

8.1 SQL Data Types

- **Numeric**: INT, SMALLINT, NUMERIC(p, s), DECIMAL(p, s), FLOAT, REAL.
- **Character**: CHAR(n) (fixed length, padded), VARCHAR(n) (variable length).
- **Date/Time**: DATE, TIME, TIMESTAMP.
- **Others**: BOOLEAN, BLOB, CLOB.

CBT Trap

- CHAR(n) is **fixed length and pads with spaces**; VARCHAR(n) stores only actual characters. Comparing 'A' (CHAR) with 'A' can surprise.

8.2 CREATE TABLE and Constraints

- CREATE TABLE defines a relation with columns, types, and constraints.
- Column/table constraints: **PRIMARY KEY, FOREIGN KEY ... REFERENCES, UNIQUE, NOT NULL, CHECK, DEFAULT**.
- Example:


```
CREATE TABLE Student(Roll INT PRIMARY KEY, Name VARCHAR(40) NOT NULL, Age INT CHECK(Age>0), Dept VARCHAR(10) DEFAULT 'CS');
```

Constraint	Enforces
PRIMARY KEY	Unique + NOT NULL
UNIQUE	Uniqueness (allows one NULL)
FOREIGN KEY	Referential integrity
NOT NULL	No NULL
CHECK	Boolean condition
DEFAULT	Default value

CBT Trap

- A **UNIQUE** column **allows NULLs** (typically one NULL in standard SQL); a **PRIMARY KEY** does **not**. Distinguish UNIQUE from PRIMARY KEY.

8.3 ALTER, DROP, TRUNCATE

- **ALTER TABLE** — ADD/DROP/MODIFY columns or constraints.
- **DROP TABLE** — removes the table **structure + data** (DDL).
- **TRUNCATE TABLE** — removes **all rows**, keeps the structure; it is **DDL, auto-commit, cannot be rolled back** (generally), and **faster** than DELETE.

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Rows (WHERE)	All rows	Table itself

Feature	DELETE	TRUNCATE	DROP
Rollback	Yes	No (usually)	No
WHERE clause	Yes	No	No
Speed	Slow	Fast	Fast

CBT Trap

- **DELETE** is **DML** (rollback-able, can have WHERE); **TRUNCATE** is **DDL** (no WHERE, resets the table, generally not rollback-able). The classic 3-way DELETE/TRUNCATE/DROP distinction.

8.4 Creating Indexes

- `CREATE INDEX idx ON Table(col);` builds an index to speed up searches; `CREATE UNIQUE INDEX` enforces uniqueness.
- Indexes speed up **SELECT/WHERE/JOIN** but **slow down INSERT/UPDATE/DELETE** (index maintenance) and use extra space.

CBT Trap

- Adding indexes is a **physical** change (physical data independence). More indexes = faster reads but **slower writes**.

9. SQL - Data Manipulation Language (DML)

9.1 SELECT, DISTINCT, Aliases

- `SELECT cols FROM table` retrieves data; `SELECT *` returns all columns.
- **DISTINCT** removes **duplicate** rows from the result.
- **Aliases** (AS) rename columns/tables for readability and self-joins.

CBT Trap

- Plain SQL `SELECT` keeps **duplicates** (bag semantics); only **DISTINCT** removes them. Relational-algebra projection removes them automatically.

9.2 WHERE Clause Operators

- Comparison: `=`, `<>` / `!=`, `<`, `>`, `<=`, `>=`.
- **BETWEEN a AND b** — inclusive range.
- **IN (set)** — membership; **LIKE** — pattern (`%` = any string, `_` = single char).
- **IS NULL** / **IS NOT NULL** — NULL test.
- Logical: **AND**, **OR**, **NOT** (precedence: NOT > AND > OR).

CBT Trap

- **BETWEEN** is **inclusive** of both endpoints. **LIKE** `'A%'` matches strings starting with A; `%` = any number of chars, `_` = exactly one char.

9.3 ORDER BY

- `ORDER BY col [ASC|DESC]` sorts the final result; default is **ASC**.
- Multiple keys sort hierarchically; can sort by column position or alias.
- **ORDER BY** is the **last** clause executed logically.

9.4 INSERT, UPDATE, DELETE

- `INSERT INTO t(cols) VALUES(...)` adds rows.
- `UPDATE t SET col=val WHERE cond` modifies rows.
- `DELETE FROM t WHERE cond` removes rows; without `WHERE` it deletes **all** rows (but keeps structure).

CBT Trap

- `UPDATE/DELETE` **without a WHERE clause** affect **every row** in the table — a common accidental-mass-change trap.

9.5 Joins in SQL

- **INNER JOIN** — only matching rows.
- **LEFT [OUTER] JOIN** — all left rows + matches (NULL padding on right).
- **RIGHT [OUTER] JOIN** — all right rows + matches.
- **FULL OUTER JOIN** — all rows from both.
- **CROSS JOIN** — Cartesian product.
- **SELF JOIN** — a table joined to itself (needs aliases).

CBT Trap

- **CROSS JOIN** with m and n rows gives **m*n** rows. A **LEFT JOIN** result has **>=** rows of the left table.

9.6 Set Operations in SQL

- **UNION** — combines + removes duplicates; **UNION ALL** — keeps duplicates (**faster**).
- **INTERSECT** — common rows; **EXCEPT** (SQL Server/Postgres) / **MINUS** (Oracle) — difference.
- Operands must be **union-compatible** (same number/type of columns).

Operator	Duplicates	Note
UNION	Removed	sorts/dedups
UNION ALL	Kept	fastest
INTERSECT	Distinct common	—
EXCEPT/MINUS	Distinct difference	non-commutative

CBT Trap

- **UNION** removes duplicates (extra sort cost); **UNION ALL** keeps them and is faster. **MINUS** (Oracle) = **EXCEPT** (others).

10. SQL - Aggregation and Grouping

10.1 Aggregate Functions

- **COUNT, SUM, AVG, MIN, MAX** operate over a set of rows returning a single value.
- All aggregates except **COUNT(*)** ignore **NULLs**.

10.2 COUNT(*) vs COUNT(column)

- **COUNT(*)** counts **all rows** including those with **NULLs**.
- **COUNT(col)** counts only rows where col is **NOT NULL**.
- **COUNT(DISTINCT col)** counts distinct non-NULL values.

CBT Trap

- **COUNT(*)** includes **NULL** rows; **COUNT(col)** skips **NULLs**; **AVG(col)** divides by the **count of non-NULL** values, not total rows.

10.3 GROUP BY

- **GROUP BY** partitions rows into groups; aggregates compute **per group**.
- Every column in **SELECT** that is **not** inside an aggregate must appear in **GROUP BY**.

10.4 HAVING vs WHERE

- **WHERE** filters **individual rows before grouping**; cannot use aggregates.
- **HAVING** filters **groups after aggregation**; can use aggregates.

Clause	Filters	Aggregates allowed?	When
WHERE	Rows	No	Before GROUP BY
HAVING	Groups	Yes	After GROUP BY

CBT Trap

- You **cannot** use an aggregate (e.g., **SUM**) in **WHERE**; aggregate conditions go in **HAVING**. This is a top-frequency MCQ.

10.5 Logical Order of Clause Evaluation

- Logical evaluation order: **FROM** -> **WHERE** -> **GROUP BY** -> **HAVING** -> **SELECT** -> **DISTINCT** -> **ORDER BY** -> **LIMIT**.
- This is why column **aliases in SELECT** usually can't be used in **WHERE** (**SELECT** runs after **WHERE**).

Memory Trick

- "**FWGHSDOL**": From, Where, Group, Having, Select, Distinct, Order, Limit.

CBT Trap

- **SELECT** is evaluated **after** **WHERE/GROUP BY/HAVING** but **before** **ORDER BY** — hence **ORDER BY** can use **SELECT** aliases while **WHERE** cannot.

11. SQL - Subqueries

11.1 Subqueries in WHERE / FROM / SELECT

- A **subquery (nested query)** is a query inside another. In **FROM** it is a **derived table/inline view**; in **SELECT** it must return a **scalar**.

11.2 IN, NOT IN, EXISTS, NOT EXISTS

- **IN** — true if value matches any in the subquery set.
- **EXISTS** — true if the subquery returns **at least one row**; returns boolean only.
- **NOT EXISTS** — used for **"for all"/division** queries.

11.3 ANY/SOME and ALL

- > ANY = greater than the **minimum**; > ALL = greater than the **maximum**.
- = ANY is equivalent to **IN**; <> ALL is equivalent to **NOT IN**.

CBT Trap

- > ALL(set) means greater than the **maximum** of the set; > ANY(set) means greater than the **minimum**. Swapping ANY/ALL is a classic trap.

11.4 Correlated Subqueries

- A **correlated subquery** references a column from the **outer query**; it is **re-evaluated once per outer row** (slower).
- **EXISTS** is usually correlated.

11.5 NOT IN with NULL Pitfall

- If the subquery of **NOT IN** returns **even one NULL**, the whole NOT IN may yield **UNKNOWN** -> **no rows returned**.
- Use **NOT EXISTS** to avoid this NULL pitfall.

CBT Trap

- **NOT IN** with a NULL in the list returns **no rows** (because of 3-valued logic); **NOT EXISTS** is safe. A subtle but frequently tested nuance.

12. SQL - Other Objects

12.1 Views

- A **view** is a **virtual table** defined by a stored query; it stores **no data** itself (unless **materialized**).
- Uses: **security** (hide columns), **simplification**, **logical data independence**.
- A view is **updatable** only if it maps unambiguously to base-table rows (no aggregates, no DISTINCT, no GROUP BY, single table generally).

CBT Trap

- A **materialized view** stores results physically (must be refreshed); a normal view is **virtual**. Views with **aggregation/GROUP BY/DISTINCT** are **not updatable**.

12.2 Triggers

- A **trigger** is procedural code that fires **automatically** on **INSERT/UPDATE/DELETE** events.
- Timing: **BEFORE** or **AFTER**; granularity: **row-level** or **statement-level**.
- Uses: enforce complex constraints, auditing, derived-column maintenance.

12.3 Stored Procedures / Functions

- **Stored procedures** are precompiled routines stored in the DB; reduce network traffic and centralize logic.
- **Functions** return a value and can be used in expressions; procedures may return via OUT params.

12.4 GRANT / REVOKE (DCL)

- **GRANT** gives privileges (SELECT, INSERT, etc.) to users/roles; **WITH GRANT OPTION** lets the grantee re-grant.
- **REVOKE** removes privileges.

CBT Trap

- **GRANT/REVOKE** are **DCL** commands (authorization), not DDL/DML. **WITH GRANT OPTION** allows cascading of granted privileges.

13. Functional Dependencies (FD)

13.1 Definition and Notation

- A **functional dependency** $X \rightarrow Y$ means: if two tuples agree on attribute set **X**, they must agree on **Y** ("X functionally determines Y"; X = determinant).
- It is a **constraint** derived from the real-world semantics, not just the current data.

13.2 Trivial vs Non-Trivial FDs

- Trivial FD**: $X \rightarrow Y$ where $Y \subseteq X$ (e.g., $AB \rightarrow A$). Always holds.
- Non-trivial FD**: $X \rightarrow Y$ where $Y \not\subseteq X$ (Y not a subset of X).
- Completely non-trivial**: X and Y are **disjoint**.

CBT Trap

- An FD is **trivial** iff the **right side is a subset of the left side**. $AB \rightarrow B$ is trivial; $A \rightarrow B$ is not.

13.3 Armstrong's Axioms

- Reflexivity**: if $Y \subseteq X$ then $X \rightarrow Y$.
- Augmentation**: if $X \rightarrow Y$ then $XZ \rightarrow YZ$.
- Transitivity**: if $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$.
- These **3** are **sound and complete**. Derived rules: **Union**, **Decomposition**, **Pseudotransitivity**.

Axiom	Rule
Reflexivity	$Y \subseteq X \Rightarrow X \rightarrow Y$
Augmentation	$X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
Transitivity	$X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$
Union (derived)	$X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$
Decomposition (derived)	$X \rightarrow YZ \Rightarrow X \rightarrow Y, X \rightarrow Z$
Pseudotransitivity	$X \rightarrow Y, WY \rightarrow Z \Rightarrow WX \rightarrow Z$

Memory Trick

- "RAT"** = Reflexivity, Augmentation, Transitivity (the 3 primary axioms).

CBT Trap

- Reflexivity, Augmentation, Transitivity** are **primary** (sound + complete); **Union, Decomposition, Pseudotransitivity** are **derived**. Don't list a derived rule as a primary axiom.

13.4 Closure of an Attribute Set (X^+)

- Attribute closure X^+** = set of all attributes functionally determined by X under FD set F.
- Algorithm: start with X; repeatedly add the RHS of any FD whose LHS is already inside X^+ , until stable.
- If **$X^+ = \text{all attributes (R)}$** , then X is a **super key**.
- Complexity: computing a closure is **$O(n^*|F|)$** roughly linear in the FDs.

CBT Trap

- To test if X is a **super key**, compute **X+** and check it equals **all attributes**. This is the master technique behind candidate-key and lossless-join problems.

13.5 Finding Candidate Keys from FDs

- Attributes appearing **only on the left** of FDs (and those never on any RHS) **must** be in **every** candidate key.
- Attributes appearing **only on the right** are **never** part of any candidate key.
- Procedure: take the mandatory-left attributes, compute closure; if it covers R, it's a candidate key; else add attributes minimally.

CBT Trap

- An attribute that appears **only on the left side** of all FDs is **always** part of **every** candidate key. Start closure from those.

13.6 Prime vs Non-Prime Attributes

- A **prime attribute** is part of **some candidate key**.
- A **non-prime (non-key) attribute** is part of **no** candidate key.

CBT Trap

- "Prime" relates to **candidate keys**, not the primary key alone. An attribute in **any** candidate key is prime.

13.7 Minimal / Canonical Cover

- A **minimal (canonical) cover** F_c is an equivalent FD set such that: every RHS is a **single attribute**, no FD is **redundant**, and no LHS has **extraneous** attributes.
- Steps: (1) **singleton RHS**; (2) remove **extraneous LHS attributes**; (3) remove **redundant FDs**.

CBT Trap

- A minimal cover must have a **single attribute on each RHS** and **no redundant FD or extraneous attribute**; multiple valid minimal covers may exist.

14. Normal Forms

14.1 Anomalies

- **Insertion anomaly**: cannot add data without other unrelated data (e.g., can't add a course with no student).
- **Deletion anomaly**: deleting a row loses unrelated info.
- **Update anomaly**: redundant data must be changed in many places, risking inconsistency.
- Normalization removes anomalies by **decomposing** relations based on FDs.

CBT Trap

- All **3** anomalies — **insertion, deletion, update** — arise from **redundancy** caused by bad dependencies; normalization is the cure.

14.2 First Normal Form (1NF)

- A relation is in **1NF** if all attribute values are **atomic (single-valued)** — no repeating groups, no multivalued attributes, no nested relations.

14.3 Second Normal Form (2NF)

- A relation is in **2NF** if it is in 1NF and has **no partial dependency** — no **non-prime attribute** depends on a **proper subset** of any candidate key.
- Partial dependency is only possible with a **composite** candidate key. If every candidate key is a single attribute, 1NF => 2NF automatically.

CBT Trap

- **Partial dependency** requires a **composite key**. A relation with all single-attribute keys is automatically in **2NF** (if 1NF).

14.4 Third Normal Form (3NF)

- A relation is in **3NF** if it is in 2NF and has **no transitive dependency** of a non-prime attribute on the key.
- Formal: for every non-trivial FD $X \rightarrow A$, either **X is a super key** OR **A is a prime attribute**.

14.5 BCNF (Boyce-Codd Normal Form)

- A relation is in **BCNF** if for every non-trivial FD $X \rightarrow A$, **X is a super key**.
- BCNF is **stricter** than 3NF: it removes the "A is prime" escape clause.
- BCNF differs from 3NF only when there is an FD where a **prime attribute depends on a non-super-key** (i.e., overlapping candidate keys).

Normal Form	Removes	Condition
1NF	Non-atomic values	Atomic only
2NF	Partial dependency	No non-prime on part of key
3NF	Transitive dependency	X super key OR A prime
BCNF	Remaining anomalies	X super key (always)

Memory Trick

- **"1-A, 2-Partial, 3-Transitive, BCNF-every determinant is a key"**.

CBT Trap

- **3NF** allows $X \rightarrow A$ when **A is prime**; **BCNF** does **not** — it requires **X to be a super key** for **every** non-trivial FD. Every BCNF relation is in 3NF, but not vice versa.

14.6 Determining the Highest Normal Form

- Find **all candidate keys** $\rightarrow$ identify **prime/non-prime** attributes $\rightarrow$ test each FD against 2NF, 3NF, BCNF in order.
- The **highest normal form** is the strictest one all FDs satisfy.

CBT Trap

- The highest normal form is the **strictest satisfied**; check **BCNF first** then fall back. A relation in BCNF is automatically in 3NF, 2NF, 1NF.

14.7 4NF and Multivalued Dependencies (Awareness)

- A **multivalued dependency (MVD)** $X \twoheadrightarrow Y$ exists when Y is independent of the rest given X.
- **4NF** requires: for every non-trivial MVD $X \twoheadrightarrow Y$, **X is a super key**. 4NF removes MVD-based redundancy.
- **5NF (PJNF)** deals with **join dependencies**.

CBT Trap

- **4NF** handles **multivalued dependencies**; **BCNF** handles only functional dependencies. Order: **1NF < 2NF < 3NF < BCNF < 4NF < 5NF**.

15. Decomposition

15.1 Lossless Join Decomposition

- A decomposition of R into R1, R2 is **lossless (non-additive)** if their natural join gives **exactly** R (no spurious tuples).
- Test: decomposition into R1, R2 is lossless iff $(R1 \cap R2) \rightarrow R1$ OR $(R1 \cap R2) \rightarrow R2$ (the common attributes form a **super key** of at least one).

CBT Trap

- A 2-way decomposition is lossless iff the **common attributes** are a **super key of at least one** sub-relation. A **lossy** decomposition creates **extra (spurious) tuples** on rejoin.

15.2 Dependency-Preserving Decomposition

- A decomposition is **dependency-preserving** if the union of FDs projected onto each sub-relation is **equivalent** to the original FD set, so constraints can be checked **without joins**.

15.3 3NF vs BCNF Trade-off

- **3NF decomposition** can **always** be both **lossless AND dependency-preserving**.
- **BCNF decomposition** is always **lossless** but **may NOT be dependency-preserving**.

Property	3NF	BCNF
Lossless join	Always achievable	Always achievable
Dependency-preserving	Always achievable	Not guaranteed

Property	3NF	BCNF
Redundancy	Some may remain	Minimal

CBT Trap

- **BCNF** may sacrifice **dependency preservation**; **3NF** can keep both. This trade-off is a top exam favorite.

16. File Organization

16.1 Records: Fixed vs Variable Length

- **Fixed-length records**: simple, fast addressing ($\text{offset} = \text{record\#} * \text{size}$), but wastes space.
- **Variable-length records**: arise from variable fields (VARCHAR) or repeating groups; need length indicators/offsets.

16.2 Heap (Unordered) Organization

- Records placed wherever space is available; **insertion is $O(1)$** (append), but **search is $O(n)$** (full scan).
- Best when retrieving most of the file or for bulk loading.

16.3 Sequential (Ordered) Organization

- Records stored **sorted** by a search key; supports **binary search $O(\log n)$** on blocks and efficient range queries.
- Insertion/deletion costly (maintain order) — use overflow blocks.

16.4 Hash File Organization

- A **hash function** maps the search-key to a **bucket**; gives **$O(1)$** average equality search.
- **Collisions** handled by chaining/overflow buckets; **poor** for **range queries**.

16.5 Blocking Factor / Block Concept

- **Block (page)** = unit of disk transfer; **blocking factor $bfr = \text{floor}(B / R)$** = records per block (B = block size, R = record size).
- Number of blocks $b = \text{ceil}(r / bfr)$ for r records.

Organization	Equality Search	Range Query	Insert
Heap	$O(n)$	$O(n)$	$O(1)$
Sequential	$O(\log n)$	Good	Costly
Hash	$O(1)$ avg	Poor	$O(1)$ avg

CBT Trap

- **Hashing** is best for **equality** but **worst for range** queries; **ordered/sequential** files and **B+ trees** are best for **range** queries. Blocking factor uses **floor**, block count uses **ceil**.

17. Indexing

17.1 Index Concept and Search Key

- An **index** is an auxiliary structure (search-key, pointer) that speeds up retrieval, like a book index.
- **Search key** = attribute(s) on which the index is built (different from a candidate key).

17.2 Primary (Clustering) vs Secondary (Non-Clustering)

- **Primary/clustering index** — built on the attribute by which the **file is physically sorted**; only **one** per table.
- **Secondary/non-clustering index** — on a non-ordering attribute; can have **many**; must be **dense**.

CBT Trap

- There can be only **ONE clustering (primary) index** per table (since a file can be physically sorted one way), but **many secondary** indexes. Secondary indexes are always **dense**.

17.3 Dense vs Sparse Index

- Dense index** — an index entry for **every** search-key value (every record).
- Sparse index** — an index entry for **some** keys (one per block); requires the data to be **ordered**.
- Sparse uses **less space** but only works on the **ordering key**.

Type	Entries	Space	Needs ordering
Dense	Every record	More	No
Sparse	One per block	Less	Yes

CBT Trap

- A **sparse index** can only exist on an **ordering/clustering** field; a **secondary index** must be **dense**. Sparse needs fewer entries (one per block).

17.4 Multilevel Index

- When an index is too large to fit in memory, build an **index on the index** (**multilevel**), reducing search to **$O(\log_{\text{fanout}} n)$** ; this idea leads to **B/B+ trees**.

17.5 B-Tree

- A **B-tree** of order **m** is a balanced multiway search tree where:
 - Every node has at most **m** children and **m-1** keys.
 - Every internal node (except root) has at least **$\text{ceil}(m/2)$** children.
 - All leaves are at the **same level** (balanced).
 - Keys + data pointers are stored in ALL nodes** (internal and leaf).
 - Search/insert/delete are **$O(\log n)$** .

CBT Trap

- In a **B-tree**, **data/record pointers exist in every node** (internal + leaf). This is the key difference from B+ trees.

17.6 B+ Tree

17.6.1 Difference from B-Tree

- In a **B+ tree**, **all data pointers are in the LEAF nodes**; internal nodes hold **only keys (index)** for routing.
- Leaf nodes are linked** in a **linked list**, enabling efficient **sequential/range** access.

17.6.2 Order / Fan-out

- For a B+ tree of order **n** (or p): an internal node has up to **n** pointers and **n-1** keys.
- Higher **fan-out** (more keys per node) => **shorter tree** => fewer disk I/Os.

17.6.3 Search / Insert / Delete

- Search always reaches a **leaf** (**$O(\log n)$**); all searches traverse the same height.
- Inserts/deletes may cause **node split** or **merge/redistribute** to keep balance.

17.6.4 Height / Node-Count Computation

- With order p and N keys, height $h \approx \lceil \log_p N \rceil$; minimizing height minimizes I/O.
- A B+ tree generally has **higher fan-out** than a B-tree of similar size (internal nodes store no data), so it is **shorter**.

Feature	B-Tree	B+ Tree
Data pointers	All nodes	Leaves only
Leaf linkage	No	Yes (linked list)
Range query	Slower	Faster (linked leaves)
Fan-out	Lower	Higher
Search cost (all keys)	Varies by level	Uniform (always to leaf)

Memory Trick

- **"B+ keeps data in leaves and links them; B-tree spreads data everywhere"**.

CBT Trap

- **B+ tree**: data ONLY in **leaves**, leaves **linked** => great for **range** queries; **B-tree**: data in **all** nodes, no leaf links. Most real DBMS indexes are **B+ trees**.

17.7 Hashing for Indexing; Extendible Hashing (Awareness)

- **Static hashing** uses a fixed number of buckets — suffers when data grows (overflow).
- **Dynamic/Extendible hashing** uses a **directory** and a **global depth**; buckets split as needed without full reorganization.
- **Linear hashing** grows buckets one at a time without a directory.

CBT Trap

- **Extendible hashing** adapts to growth using a **directory + global/local depth**; **static hashing** causes overflow chains when the file grows. Hashing is still **poor for range** queries.

18. Transactions

18.1 Transaction Concept; Read/Write Operations

- A **transaction** is a **logical unit of work** — a sequence of operations executed as a single **all-or-nothing** unit.
- Basic operations: **read(X)** brings data item X from disk to memory; **write(X)** writes the buffer value back; **commit** makes changes permanent; **abort/rollback** undoes them.

18.2 ACID Properties

- **Atomicity** — all operations execute or **none** do ("all-or-nothing"); ensured by the **recovery/transaction manager**.
- **Consistency** — a transaction takes the DB from one **consistent state to another**; responsibility shared with the application.
- **Isolation** — concurrent transactions appear to run **serially**; ensured by **concurrency control**.
- **Durability** — once committed, changes **survive crashes**; ensured by **recovery (logs)**.

Property	Ensured By
Atomicity	Transaction/recovery manager
Consistency	Application + constraints
Isolation	Concurrency-control manager
Durability	Recovery manager (logs)

Memory Trick

- **"ACID"** = Atomicity, Consistency, Isolation, Durability.

CBT Trap

- **Atomicity** = all-or-nothing; **Durability** = survives crashes. **Isolation** is enforced by **concurrency control**; **Durability** by **recovery**. Don't mix which manager handles which.

18.3 Transaction States

- **Active** -> initial state while executing.
- **Partially committed** -> after the **final operation** but before commit confirmed.
- **Committed** -> after successful completion (durable).
- **Failed** -> after an error/abort.
- **Terminated (Aborted)** -> rolled back; can be restarted or killed.

CBT Trap

- A transaction goes **Active -> Partially committed -> Committed** on success, or **Active -> Failed -> Aborted** on failure. "Partially committed" precedes "committed".

18.4 Serial vs Concurrent Schedules

- A **schedule** is the chronological order of operations from concurrent transactions.
- **Serial schedule** — transactions execute one after another, no interleaving; always **consistent** but low throughput.
- **Concurrent schedule** — operations interleave; improves throughput/resource use but risks anomalies.

18.5 Need for Concurrency

- Concurrency improves **CPU/disk utilization** and **throughput** and reduces **average response/wait time** (long and short transactions coexist).

19. Concurrency Problems

19.1 Lost Update

- Lost update (write-write)**: two transactions read then write the same item; one update **overwrites** the other and is lost.

19.2 Dirty Read

- Dirty read (write-read)**: a transaction reads **uncommitted** data of another which later **aborts** — reading invalid data.

19.3 Unrepeatable Read

- Unrepeatable read (read-write)**: a transaction reads the same item **twice** and gets **different values** because another committed an update in between.

19.4 Phantom Read

- Phantom read**: a re-executed range query returns **additional/fewer rows** because another transaction inserted/deleted matching rows.

Problem	Pattern	Cause
Lost update	W-W	Overwrite
Dirty read	W-R	Reads uncommitted
Unrepeatable read	R-W	Value changed
Phantom	R-insert	New rows appear

CBT Trap

- Dirty read** = reading **uncommitted** data; **unrepeatable read** = same row different value; **phantom** = new/removed **rows** in a range. Phantom is about row **set membership**, not value.

20. Serializability

20.1 Conflict Operations and Conflict Serializability

- Two operations **conflict** if they belong to **different transactions**, access the **same data item**, and **at least one is a write**.
- The **3** conflict types: **W-W**, **R-W**, **W-R** (R-R never conflicts).
- A schedule is **conflict serializable** if it can be transformed into a **serial schedule** by swapping **non-conflicting** adjacent operations.

CBT Trap

- Read-Read never conflicts**. A conflict needs the same item, different transactions, and **≥ 1 write**.

20.2 Precedence (Serialization) Graph

- Build a **precedence graph**: a node per transaction; an edge $T_i \rightarrow T_j$ whenever an operation of T_i conflicts with and precedes one of T_j .
- A schedule is **conflict serializable iff the precedence graph is ACYCLIC**.
- A topological sort of the acyclic graph gives an equivalent serial order.

Memory Trick

- "Cycle = NOT serializable; Acyclic = serializable"**.

CBT Trap

- Conflict serializable $\Leftrightarrow$ precedence graph has NO cycle**. A **cycle** means it is NOT conflict serializable. This is the highest-frequency transaction MCQ.

20.3 View Serializability

- A schedule is **view serializable** if it is **view-equivalent** to a serial schedule (same **initial reads**, same **write-read (read-from)** pairs, same **final writes**).
- Conflict serializable $\subset$ View serializable** — every conflict-serializable schedule is view serializable, but not vice versa.
- View serializability allows **blind writes**; testing it is **NP-complete**.

CBT Trap

- View serializability is more general** (a superset) than **conflict serializability** and permits **blind writes**; its test is **NP-complete**, while conflict serializability is checked by an easy cycle test.

20.4 Recoverability

- Recoverable schedule**: a transaction commits only **after** all transactions it **read from** have committed (prevents committing on dirty data).
- Cascadeless (ACR) schedule**: transactions read only **committed** data — avoids **cascading rollback**.
- Strict schedule**: no transaction may **read or write** an item until the last writer **commits/aborts**; simplifies recovery.
- Hierarchy: **Strict $\subset$ Cascadeless $\subset$ Recoverable**.

CBT Trap

- **Cascading rollback** (one abort forcing others) is avoided by **cascadeless** schedules. Strictness is the strongest; the chain is **Strict** $\subset$ **Cascadeless** $\subset$ **Recoverable**.

21. Concurrency Control Protocols

21.1 Lock-Based Protocols

21.1.1 Shared and Exclusive Locks

- **Shared lock (S)** — for **reading**; multiple transactions may hold S on the same item.
- **Exclusive lock (X)** — for **writing**; only **one** transaction may hold it; conflicts with all.

	S held	X held
Request S	Yes (compatible)	No
Request X	No	No

21.1.2 Two-Phase Locking (2PL)

- **2PL** guarantees **conflict serializability** via two phases:
- **Growing phase** — locks are **acquired**, none released.
- **Shrinking phase** — locks are **released**, none acquired.
- The point of the last lock acquisition is the **lock point**.
- Basic 2PL can still cause **deadlocks** and **cascading rollbacks**.

21.1.3 Strict and Rigorous 2PL

- **Strict 2PL** — holds all **exclusive (write)** locks until **commit/abort**; avoids cascading rollback.
- **Rigorous 2PL** — holds **all** locks (S and X) until commit; even stronger.

Memory Trick

- **"2PL = Grow then Shrink, never grow again"**.

CBT Trap

- **Basic 2PL** guarantees serializability but **NOT** deadlock-freedom and **NOT** cascadelessness. **Strict 2PL** holds **X-locks** till commit (cascadeless); **Rigorous** holds **all** locks till commit.

21.2 Deadlock Handling

- A **deadlock** occurs when transactions wait for each other's locks in a **cycle**.
- **4** Coffman conditions: **mutual exclusion**, **hold and wait**, **no preemption**, **circular wait**.
- **Wait-die (non-preemptive)**: an **older** transaction waits; a **younger** one is **killed (dies)** when requesting a lock held by an older one.
- **Wound-wait (preemptive)**: an **older** transaction **wounds (preempts/aborts)** a younger holder; a younger one **waits**.

Scheme	Requesting older	Requesting younger	Type
Wait-die	waits	dies	Non-preemptive
Wound-wait	wounds younger	waits	Preemptive

CBT Trap

- In both schemes the **older** transaction is favored (never killed). **Wait-die**: older **waits**; **Wound-wait**: older **preempts**. Memorize "older survives".

21.3 Timestamp Ordering Protocol

- Each transaction gets a unique **timestamp TS(Ti)**; conflicting operations are ordered by timestamp (serializable in timestamp order; **deadlock-free** but may cause **starvation**/restarts).
- Each item keeps **read_TS** and **write_TS**. A late operation that violates order causes a **rollback**.
- **Thomas Write Rule**: ignores an **obsolete (outdated) write** instead of rejecting it, allowing more concurrency (view-serializable).

CBT Trap

- **Timestamp ordering** is **deadlock-free** (no waiting) but can cause **starvation** via repeated restarts. **Thomas write rule ignores** obsolete writes rather than aborting.

21.4 Optimistic / Validation-Based

- **Optimistic concurrency control** assumes conflicts are rare; runs in **3** phases: **Read, Validation, Write**.
- Conflicts detected only at **validation**; suited to **read-heavy**, low-conflict workloads.

CBT Trap

- Optimistic CC has **3** phases — **Read, Validation, Write** — and checks conflicts only at the **validation** phase; ideal when conflicts are **infrequent**.

22. Recovery

22.1 Failure Types

- **Transaction failure** (logical error / system-enforced abort).
- **System crash** (hardware/software/power; loses volatile memory but disk intact).
- **Disk failure** (media failure; loses non-volatile storage).

22.2 Log-Based Recovery; Deferred vs Immediate Update

- A **log** records every write as **<Ti, X, old_value, new_value>** on stable storage.
- **Deferred update**: writes applied to DB **only after commit** -> needs only **REDO**.
- **Immediate update**: writes applied **before commit** too -> needs **UNDO** and **REDO**.

Technique	On disk before commit?	Recovery
Deferred	No	REDO only
Immediate	Yes	UNDO + REDO

CBT Trap

- **Deferred update** = **REDO only**; **Immediate update** = **UNDO + REDO**. A committed transaction is **REDONE**; an uncommitted one is **UNDONE**.

22.3 Checkpoints; Write-Ahead Logging

- A **checkpoint** periodically flushes the log + buffers, limiting how far back recovery must scan.
- **Write-Ahead Logging (WAL)**: the **log record must reach stable storage BEFORE the data** it describes; enforces **atomicity + durability**.

CBT Trap

- **WAL** requires the **log** to be written **before** the corresponding data block. Checkpoints reduce **recovery time** by bounding the redo/undo scan.

22.4 Shadow Paging (Awareness)

- **Shadow paging** keeps a **shadow (old) page table** and a **current page table**; commit atomically switches to the current table — **no log needed**, but poor for concurrency and causes fragmentation.

CBT Trap

- **Shadow paging** avoids a log by maintaining **shadow + current page tables**; its drawback is **fragmentation** and difficulty with **concurrent** transactions.

23. NoSQL Databases (Dedicated Section)

23.1 What is NoSQL and Why It Emerged

- **NoSQL** = "Not Only SQL" — a class of **non-relational**, often **schema-less**, **horizontally scalable** databases built for **Big Data** and high-velocity web workloads.
- Drivers: **huge data volume**, **high write throughput**, **flexible/evolving schema**, **horizontal scale-out** on commodity hardware, and **distribution**.
- NoSQL trades some **strong consistency** and **joins** for **scalability** and **availability**.
- The **4** main categories: **key-value**, **document**, **column-family (wide-column)**, and **graph** stores.

Category	Data Unit	Example	Best For
Key-Value	key -> opaque value	Redis , DynamoDB	Caching, sessions
Document	JSON/BSON document	MongoDB , CouchDB	Content, catalogs
Column-family	column families	Cassandra , HBase	Time-series, big writes
Graph	nodes + edges	Neo4j , JanusGraph	Relationships, networks

Memory Trick

- "**KDCG**" = Key-value, Document, Column-family, Graph (the 4 NoSQL types).

CBT Trap

- NoSQL means "**Not Only SQL**", not "No SQL". It is not always faster — it trades **ACID/joins** for **scale and flexibility**.

23.2 Key-Value Stores

- Simplest model: data stored as (**key**, **value**) pairs; value is an **opaque blob** the DB does not interpret.
- Access is by **key only** — **O(1)** get/put; no rich querying on values.
- Examples: **Redis** (in-memory, default port **6379**), **Amazon DynamoDB**, **Riak**, **Memcached**.
- Best uses: **caching**, **session storage**, **shopping carts**, **leaderboards**.

CBT Trap

- Key-value stores allow lookup **only by key (O(1))**; you **cannot** query by value efficiently. **Redis** default port is **6379**.

23.3 Document Stores

- Store **semi-structured documents** (**JSON**, **BSON**, **XML**); each document is self-describing and can have a **different schema**.
- Support **querying on fields** inside documents and **secondary indexes** — richer than key-value.
- Examples: **MongoDB** (BSON, default port **27017**), **CouchDB**, **Couchbase**.
- A document store is essentially a key-value store where the **value is a queryable document**.

CBT Trap

- **MongoDB** stores **BSON** (Binary JSON) and listens on port **27017**. Documents in one collection **need not share the same schema** (schema-flexible).

23.4 Column-Family (Wide-Column) Stores

- Data organized into **column families**; rows can have **different columns**; values stored **column-wise** for efficient large-scale reads/writes.
- Inspired by **Google Bigtable**; examples: **Apache Cassandra**, **HBase**, **ScyllaDB**.
- Excellent for **write-heavy**, **time-series**, and **analytical** workloads at massive scale.
- **Cassandra** uses a **ring** architecture with **consistent hashing** and is **AP** (tunable consistency); default client port **9042** (CQL).

CBT Trap

- Column-family stores (**Cassandra**, **HBase**) descend from **Google Bigtable** and store data **by column family**, not as relational rows. **Cassandra** is **masterless (peer-to-peer)**.

23.5 Graph Databases

- Model data as **nodes (entities)**, **edges (relationships)**, and **properties**; relationships are **first-class citizens**.
- Excel at **traversal/relationship** queries (social networks, fraud detection, recommendations) where SQL joins would be costly.
- Examples: **Neo4j** (query language **Cypher**), **Amazon Neptune**, **JanusGraph**.

CBT Trap

- **Graph databases** make **relationships** first-class and use traversal (e.g., **Cypher** in **Neo4j**); they outperform relational **joins** for deep/connected queries.

23.6 CAP Theorem (Brewer's Theorem)

- **CAP theorem** (Eric Brewer, **2000**): a distributed system can simultaneously guarantee at most **2** of the **3**: **Consistency**, **Availability**, **Partition tolerance**.
- **Consistency (C)** — every read sees the **most recent write** (all nodes agree).
- **Availability (A)** — every request gets a **non-error response** (no guarantee it is the latest).
- **Partition tolerance (P)** — the system keeps working despite **network partitions** (dropped/delayed messages).
- Since network **partitions are unavoidable** in distributed systems, real systems choose between **CP** and **AP**.

Choice	Sacrifices	Examples
CP	Availability	HBase , MongoDB (default), Redis
AP	Consistency	Cassandra , DynamoDB , CouchDB
CA	Partition tolerance (only non-distributed)	Single-node RDBMS

Memory Trick

- "**Pick 2 of 3**: C, A, P — but P is mandatory when distributed, so really CP vs AP".

CBT Trap

- You can only pick **2** of **C, A, P**. In a real distributed system **P is non-negotiable**, so the real trade-off is **CP vs AP**. A **CA** system is effectively **single-node**.

23.7 BASE vs ACID

- **BASE** = **Basically Available**, **Soft state**, **Eventual consistency** — the consistency model favored by **AP** NoSQL systems.
- **Basically Available** — system stays available (responds) even during failures.
- **Soft state** — state may change over time even without input (due to propagation).
- **Eventual consistency** — replicas **converge** to the same value **eventually** if no new writes occur.

- **ACID** (relational) favors **strong consistency**; **BASE** favors **availability + scale**.

Aspect	ACID (RDBMS)	BASE (NoSQL)
Consistency	Strong, immediate	Eventual
Availability	Lower under partition	High
Transactions	Strong, multi-row	Limited
Scale	Vertical (scale-up)	Horizontal (scale-out)
Use	Banking, OLTP	Web scale, big data

Memory Trick

- "**ACID is an acid (strict)**; **BASE is a base (relaxed)**" — chemistry opposites mirror consistency strictness.

CBT Trap

- **BASE** = **Basically Available, Soft state, Eventual consistency** — it is the **opposite philosophy** to **ACID**. Eventual consistency means replicas converge **eventually**, not instantly.

23.8 Sharding (Horizontal Partitioning)

- **Sharding** = **horizontal partitioning** of data across multiple servers (**shards**); each shard holds a **subset of rows**.
- A **shard key** determines which shard a record goes to.
- Strategies: **range-based** sharding, **hash-based** sharding, and **directory/list-based** sharding.
- **Consistent hashing** minimizes data movement when nodes are added/removed (used by **Cassandra**, **DynamoDB**).
- Benefits: **horizontal scale**, **parallelism**; risks: **hot spots**, costly **cross-shard joins**, rebalancing complexity.

Sharding Strategy	Mechanism	Risk
Range-based	Contiguous key ranges	Hot spots / skew
Hash-based	Hash of shard key	No range queries
Consistent hashing	Hash ring	Best for elasticity

CBT Trap

- **Sharding** is **horizontal** partitioning (splitting **rows** across nodes), NOT vertical (splitting columns). **Hash-based** sharding spreads load but **breaks range queries**; **consistent hashing** minimizes reshuffling.

23.9 Replication

- **Replication** = keeping **copies** of data on multiple nodes for **availability**, **fault tolerance**, and **read scaling**.
- **Master-slave (primary-replica)**: one node accepts **writes**; replicas serve **reads**; risk of **replication lag** (stale reads).
- **Master-master (multi-primary)**: multiple nodes accept writes; risk of **write conflicts** needing resolution.
- **Peer-to-peer (masterless)**: all nodes equal (e.g., **Cassandra**); uses **quorum** reads/writes.
- **Synchronous** replication = strong consistency, higher latency; **asynchronous** = lower latency, possible data loss.
- **Quorum**: with N replicas, consistency holds if **R + W > N** (R = read quorum, W = write quorum).

Model	Writes	Pro	Con
Master-slave	One node	Simple, read-scaling	Lag, write bottleneck
Master-master	Many nodes	Write availability	Conflicts
Peer-to-peer	All nodes	No SPOF	Complex coordination

CBT Trap

- Strong consistency in a quorum system requires $R + W > N$. **Synchronous** replication is consistent but slow; **asynchronous** risks **losing recent writes** on failure. Replication is for **availability**; sharding is for **capacity/scale**.

23.10 SQL vs NoSQL Summary

Dimension	SQL (Relational)	NoSQL
Schema	Fixed, predefined	Dynamic/flexible
Scaling	Vertical	Horizontal (sharding)
Transactions	Full ACID	Often BASE
Query language	SQL (standard)	Varies (CQL, Cypher, APIs)
Joins	Native	Limited / app-side
Data model	Tables	KV/Doc/Column/Graph
Consistency	Strong	Often eventual

CBT Trap

- NoSQL is not a replacement for SQL — choose by workload: **strong consistency + complex joins** -> **SQL**; **massive scale + flexible schema** -> **NoSQL**.

24. CIL MT Exam Focus / Practice Checklist

24.1 Candidate Key Finding from FD Sets

- Identify attributes on the **left-only** (must be in every key), compute **closure**, extend minimally until closure = R.
- Number of candidate keys can be ≥ 1 ; an attribute on the **right-only** is never in any key.

24.2 Identifying the Highest Normal Form

- Steps: find candidate keys $\rightarrow$ mark prime/non-prime $\rightarrow$ test **2NF (partial)**, **3NF (transitive)**, **BCNF (every determinant a super key)**.

24.3 Lossless Decomposition Test

- For R1, R2: lossless iff **$R1 \cap R2 \rightarrow R1$** or **$R1 \cap R2 \rightarrow R2$** (common attributes are a super key of one).

24.4 Conflict-Serializability via Precedence Graph

- Draw nodes/edges from conflicting ops; **acyclic $\Rightarrow$ conflict serializable**; a cycle $\Rightarrow$ not.

24.5 SQL Query Output / Equivalence

- Watch **NULL handling**, **DISTINCT**, **GROUP BY/HAVING**, **join row counts**, **NOT IN with NULL**.

24.6 Relational Algebra $\leftrightarrow$ SQL Translation

- $\sigma \rightarrow$ WHERE, $\pi \rightarrow$ SELECT, $\bowtie \rightarrow$ JOIN, $\times \rightarrow$ CROSS JOIN, $\div \rightarrow$ NOT EXISTS pattern.

24.7 B/B+ Tree Order, Height, Node-Count

- Order p: internal node has up to **p pointers, p-1 keys**; compute height as **$\text{ceil}(\log_p N)$** ; B+ tree leaves are **linked**.

24.8 Counting Tuples in Joins/Set Operations

- Cross product = **$m \cdot n$** rows; UNION (distinct) $\leq$ **$m+n$** ; INTERSECT $\leq$ $\min(m,n)$; natural join depends on matching keys.

24.9 Definitions and One-Liners

- Memorize **ACID**, **CAP**, **BASE**, key types, anomaly types, normal-form conditions, lock types, and serializability tests.

Quick Numeric Recall Table

Item	Value
ACID properties	4
Coffman deadlock conditions	4
ANSI/SPARC schema levels	3
Armstrong primary axioms	3
CAP choose at most	2 of 3

Item	Value
NoSQL categories	4
Conflict types (W-W,R-W,W-R)	3
Optimistic CC phases	3
Redis port	6379
MongoDB port	27017
Cassandra CQL port	9042

CBT Trap

- Lock in the numbers: **ACID=4**, **CAP=2 of 3**, **schema levels=3**, **Armstrong axioms=3**, **NoSQL types=4**, **deadlock conditions=4**. These pure-recall items appear almost every paper.